Jenkin – Docker – Kubernetes (CI-CD Demo)

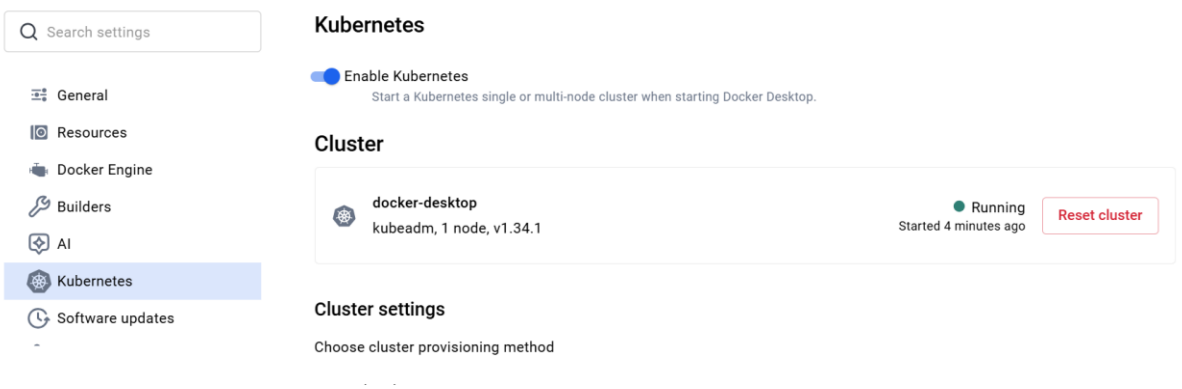
Step1

Install Docker Desktop

Step2

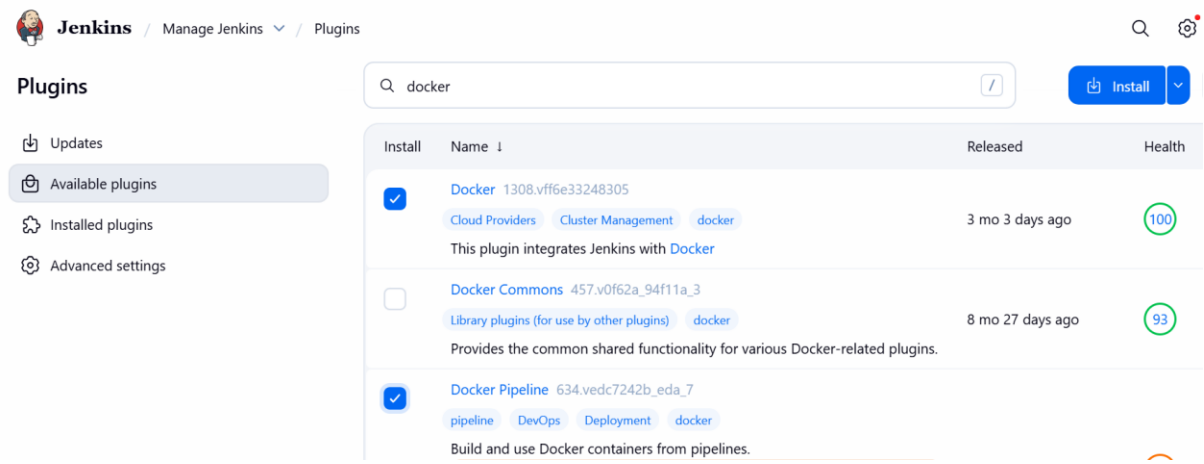
Enable Kubernetes

1. **Open the Docker Dashboard**: Launch the Docker Desktop application on your system.
2. **Navigate to Settings**: Click the gear icon (Settings or Preferences) in the top-right or sidebar of the Docker Dashboard.
3. **Select the Kubernetes Tab**: In the Settings menu, find and select the **Kubernetes** tab.
4. **Enable Kubernetes**: Check the box labeled **Enable Kubernetes**.
5. **Apply & Restart**: Click the **Apply & Restart** button at the bottom of the settings window. Docker Desktop will then download the necessary Kubernetes components and start a single-node cluster. This may take a few minutes the first time



Step 3

In Jenkin, enable two plugins (Docker and Docker Pipeline) and Install them



Step 4 - Create Simple Java Application

Create a folder in local machine: simple-java-app

Create Java File **App.java**

```
public class App {
    public static void main(String[] args) throws Exception {
        System.out.println("Java CI/CD Application Started...");
        while (true) {
            Thread.sleep(5000);
            System.out.println("Hello from Java CI/CD Pipeline!");
        }
    }
}
```

Create Maven File **pom.xml**

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         http://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>
    <groupId>com.demo</groupId>
    <artifactId>simple-java-app</artifactId>
    <version>1.0</version>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.10.1</version>
                <configuration>
                    <source>17</source>
```

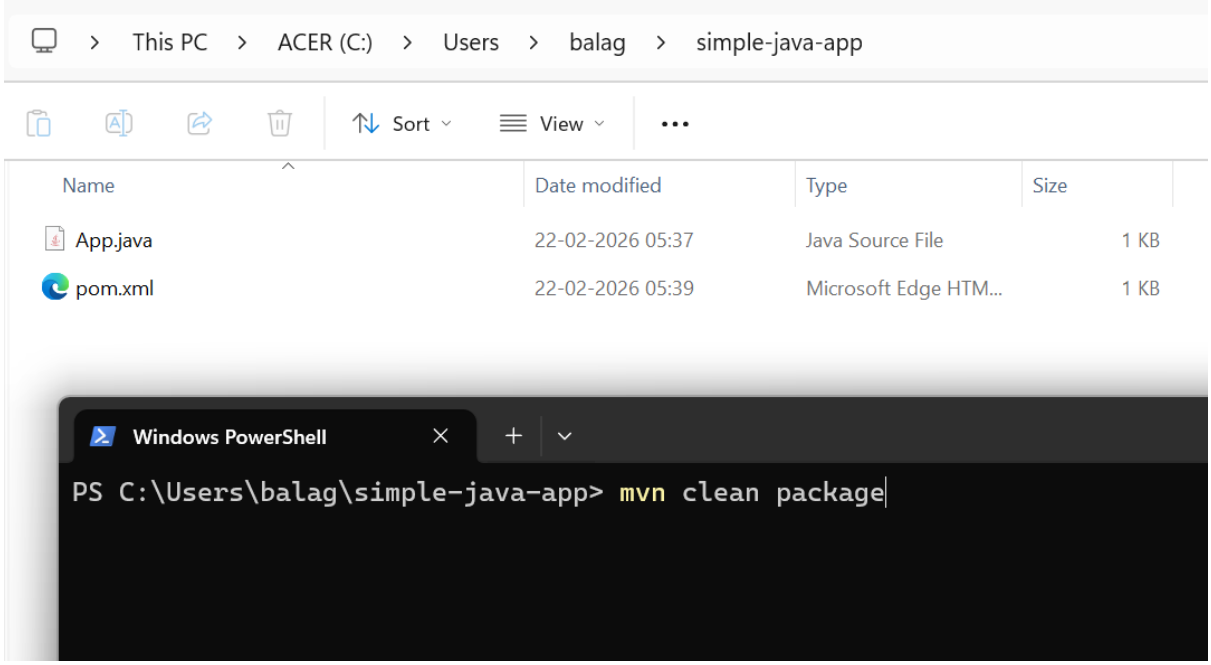
```
            <target>17</target>
          </configuration>
        </plugin>
      </plugins>
    </build>

</project>
```

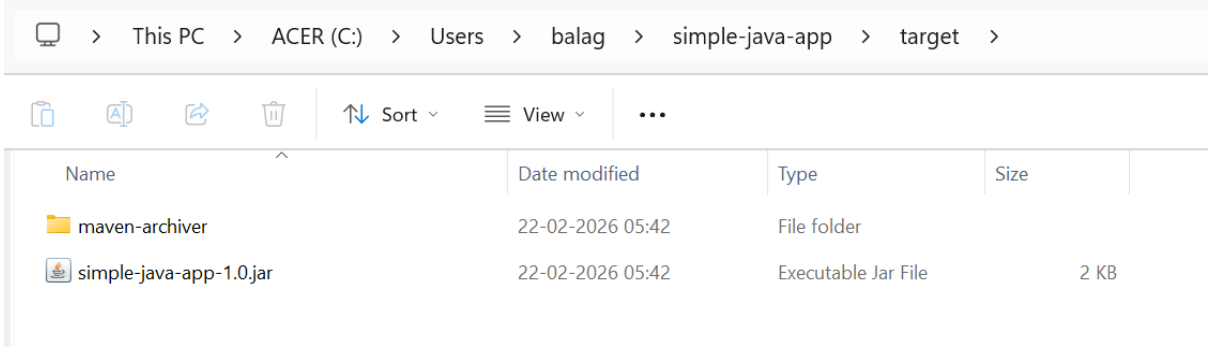
Step 5 - Build Locally (Test)

Inside project folder:

```
mvn clean package
```

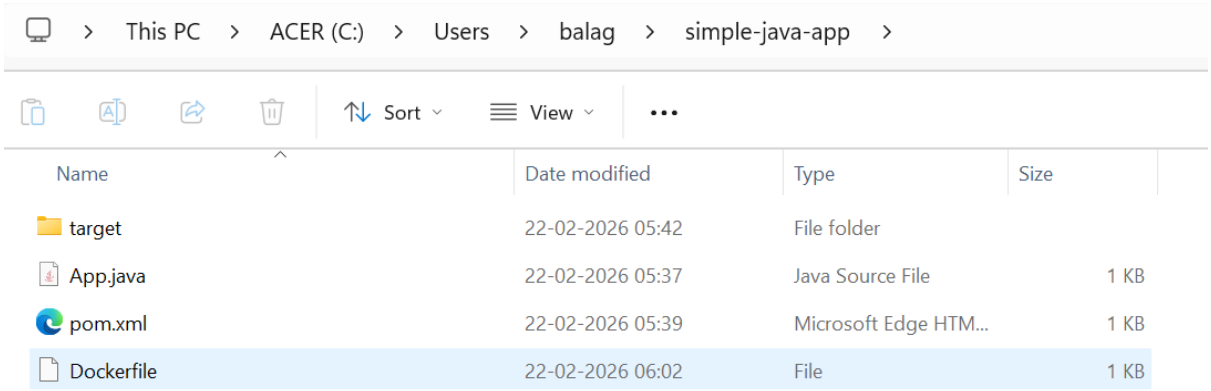


It generates: target/simple-java-app-1.0.jar



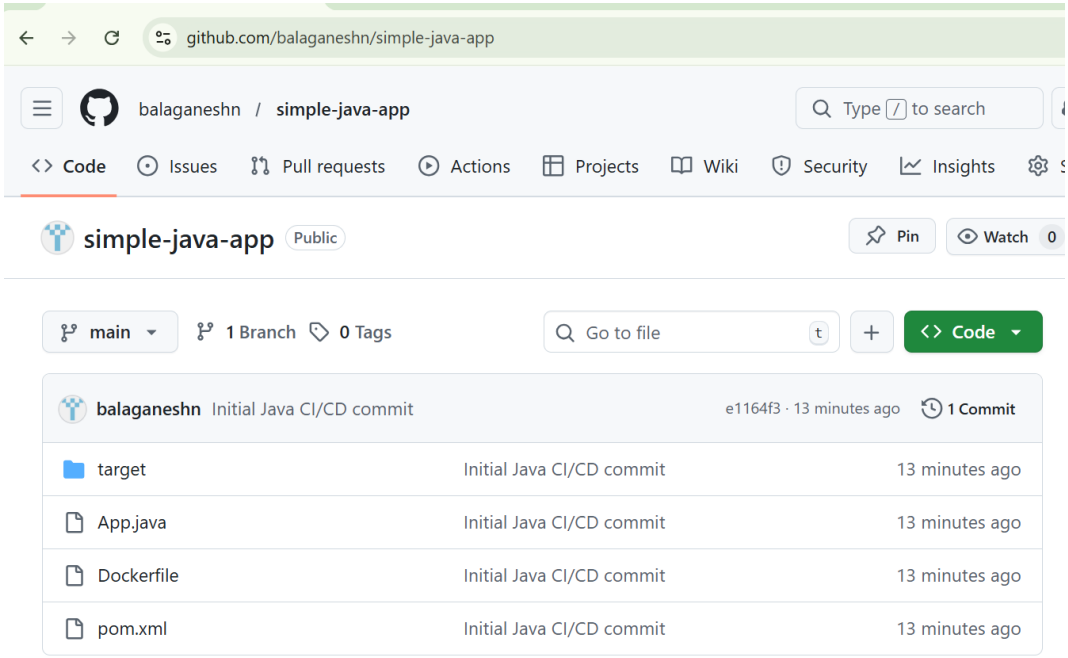
Step 6 - Create Dockerfile Dockerfile

```
FROM eclipse-temurin:17-jdk
WORKDIR /app
COPY target/simple-java-app-1.0.jar app.jar
CMD ["java", "-jar", "app.jar"]
```



Step 7 - Push Code to GitHub (Before that create a public repository in the Github)

```
git init
git add .
git commit -m "Initial Java CI/CD commit
git branch -M main
git remote add origin https://github.com/your-reponame/simple-java-app.git
git push -u origin main
```



Step 8 - Create Kubernetes Deployment File **deployment.yaml**

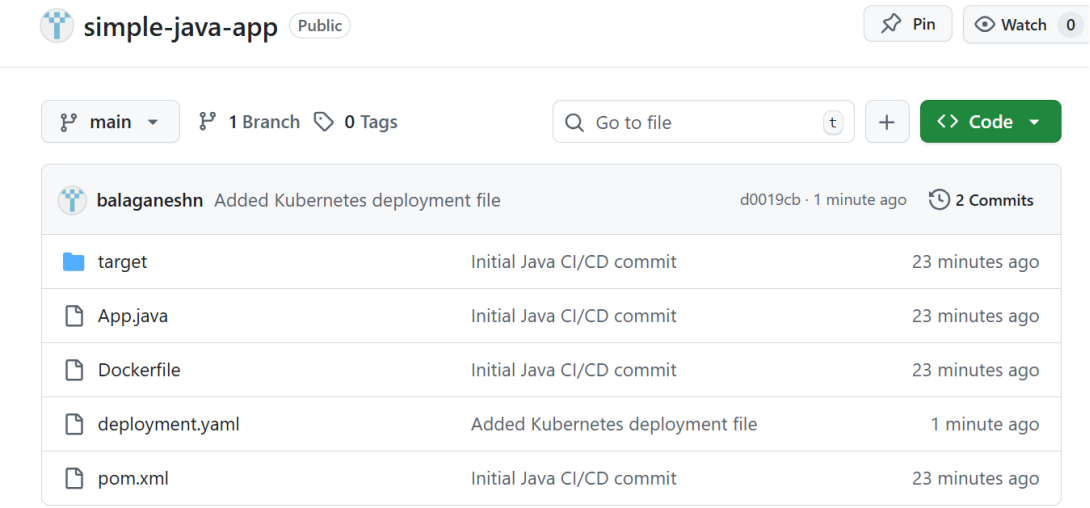
```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: java-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: java-app
  template:
    metadata:
      labels:
        app: java-app
    spec:
      containers:
      - name: java-app
        image: <dockerhub-username>/java-app:latest
---
apiVersion: v1
kind: Service
metadata:
  name: java-service
spec:
  type: NodePort
  selector:
    app: java-app
  ports:
    - port: 80
      targetPort: 8080
      nodePort: 30008
```

After creating it, Push it to GitHub:

git add deployment.yaml

git commit -m "Added Kubernetes deployment file"

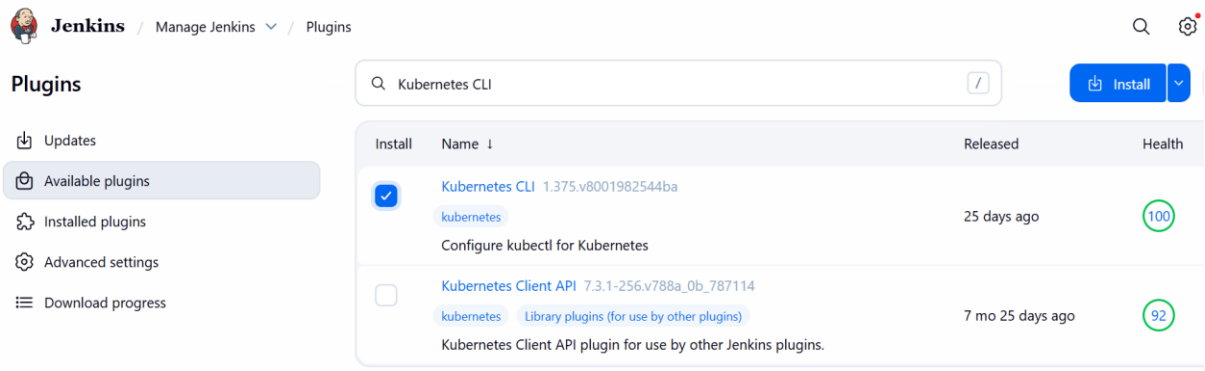
git push



Step 9 - Configure Jenkins

Run Jenkins

Install Kubernetes CLI plugin



Step 10 - Add DockerHub Credentials

In CI/CD:

1. Jenkins builds Docker image
2. Jenkins pushes image to Docker Hub
3. Kubernetes pulls image from Docker Hub

Without credentials → Push will fail

Steps

Manage Jenkins
↓
Manage Credentials
↓
(System)
↓
Global credentials (unrestricted)

Add Credentials

Fill like this:

Field	**Value**
Kind	Username with password
Scope	Global
Username	**your DockerHub username**
Password	**your DockerHub password**
ID	dockerhub-creds
Description	DockerHub Login

Click **Create**

Step 11 - Create Jenkins Pipeline Job

New Item → Pipeline → Name: **Java-CICD**

Pipleline Script:

```
pipeline {
    agent any

    environment {
        DOCKER_IMAGE = " yourdockerhubusername/java-app:latest"
    }
```

```
stages {
    stage('Clone Repository') {
        steps {
            git branch: 'main',
                url: 'https://github.com/your-reponame/simple-java-app.git'
        }
    }

    stage('Build with Maven') {
        steps {
            bat 'mvn clean package'
        }
    }

    stage('Build Docker Image') {
        steps {
            bat "docker build -t %DOCKER_IMAGE% ."
        }
    }

    stage('Push to DockerHub') {
        steps {
            withCredentials([usernamePassword(
                credentialsId: 'dockerhub-creds',
                usernameVariable: 'DOCKER_USER',
                passwordVariable: 'DOCKER_PASS'
            )]) {
                bat """
                    docker login -u %DOCKER_USER% -p %DOCKER_PASS%
                    docker push %DOCKER_IMAGE%
                """
```

```
            }
        }
    }

    stage('Deploy to Kubernetes') {
        steps {
            bat 'kubectl apply -f deployment.yaml'
        }
    }
}

post {
    success {
        echo 'CI/CD Pipeline executed successfully!'
    }
    failure {
        echo 'Pipeline failed. Please check logs.'
    }
}
}
```

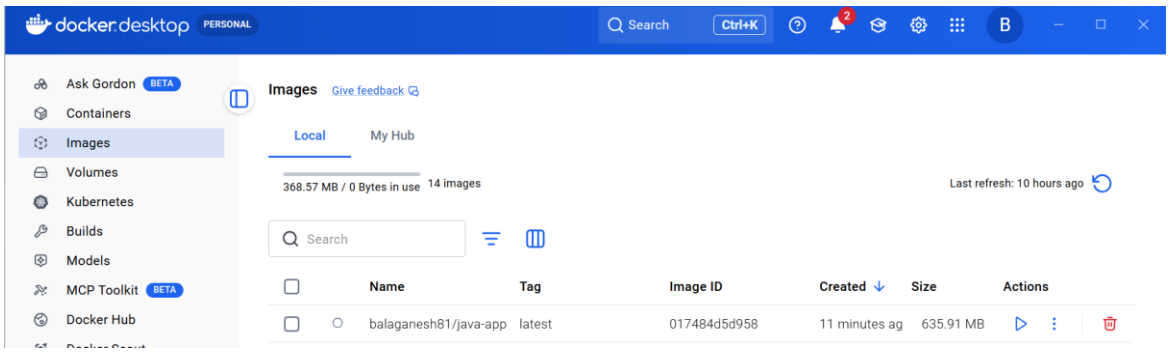
Click → Save → Build Now

```
deployment.apps/java-app created
service/java-service created
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
CI/CD Pipeline executed successfully!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

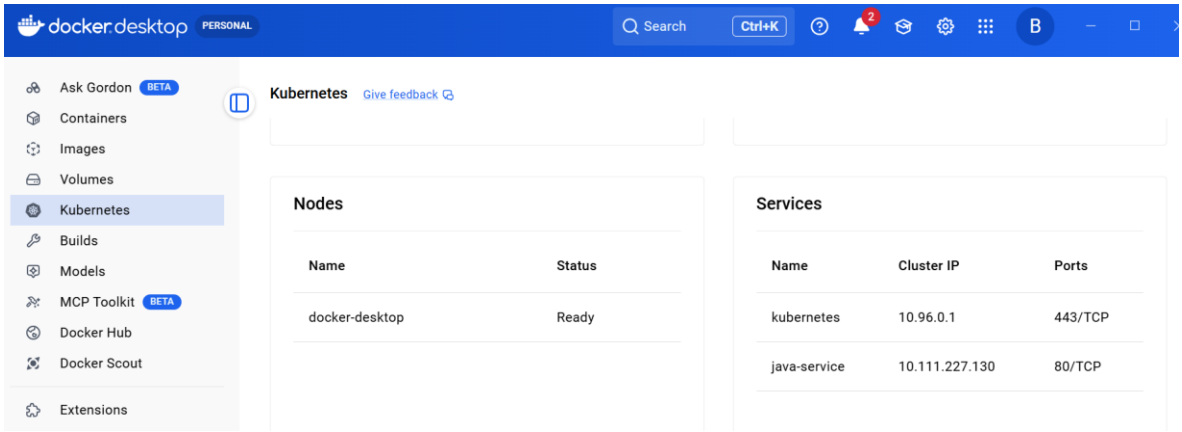
Above console output is the success message

Step 12 - Verify Docker and Kubernetes Deployment

Docker Image



Kubernetes



Run this command to verify in command prompt,

kubectl get svc

```
PS C:\Users\balag\simple-java-app> kubectl get svc
NAME           TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
java-service   NodePort    10.111.227.130   <none>        80:30008/TCP   5m28s
kubernetes     ClusterIP   10.96.0.1        <none>        443/TCP        169m
PS C:\Users\balag\simple-java-app>
```

Summary

CI/CD Flow Explanation

CI

✓ Developer pushes Java code
✓ Jenkins builds using Maven
✓ Docker image created

CD

✓ Image pushed to DockerHub
✓ Kubernetes automatically deploys
✓ Pods running inside cluster

- Jenkins clones repo
- Maven compiles Java
- Docker creates container
- DockerHub stores image
- Kubernetes pulls image
- Pods created & scaled